# SRE / DevOps / AWS / Kubernetes Interview Guide

Merged and de-duplicated interview preparation guide generated from uploaded documents. Total unique sections: **128**.

**1.** 1. How do you design SLOs for a complex distributed system?
Answer:
Start with user journey (critical paths)
Define SLIs:
Availability (success rate)
Latency (p95/p99)
Break into per-service SLOs (error budget slicing)
Avoid over-tight SLOs → causes alert fatigue
Align with business impact
? SME insight:
“SLOs should reflect user pain, not system perfection.”

**2.** 2. Scenario: Frequent production incidents despite good monitoring
Answer:
Problem is not visibility → it's signal quality
Actions:
Audit alerts → remove noisy/non-actionable
Introduce SLO-based alerting
Improve runbooks
Add tracing for unknown failures
Conduct postmortem trend analysis
? Root issue:
Monitoring exists, but observability maturity is low

**3.** 3. How do you prevent cascading failures?
Answer:
Timeouts (strict, not infinite)
Retries with exponential backoff
Circuit breakers
Bulkheads (isolation)
Load shedding
? Example:
If downstream service is slow → fail fast instead of blocking threads

**4.** 4. Scenario: Sudden spike in latency, no errors
Answer:
Check:
CPU / memory / saturation
Thread pool exhaustion
DB slow queries
Network latency
Likely cause:
Resource contention or queue buildup
? SME angle:
Latency without errors often means system is degrading, not failing yet

**5.** 5. How do you balance reliability vs feature velocity?
Answer:
Use error budget policy
If budget healthy → allow releases
If exhausted → freeze deployments
? This removes emotional debates and makes decisions data-driven

**6.** 6. Scenario: Deployment caused partial outage (only some users impacted)
Answer:
Likely:
Canary issue
AZ-specific issue
Config inconsistency
Steps:
Compare healthy vs impacted instances
Rollback or stop rollout
Check:
feature flags
infra differences
? SME thinking:
“Partial failures are harder than full outages.”

**7.** 7. How do you improve MTTR at scale?
Answer:
Standardized dashboards
Service ownership clarity
Runbooks
Auto-remediation
Game days / incident drills
Centralized logging + tracing
? Key:
Reduce time to detect + time to diagnose

**8.** 8. Scenario: Alerts are too noisy and engineers ignore them
Answer:
Remove non-actionable alerts
Use:
SLO-based alerts
Multi-window burn rate alerts
Add alert severity levels
Deduplicate alerts
? Golden rule:
“If an alert doesn’t require action → it should not exist”

**9.** 9. How do you design for multi-region high availability?
Answer:
Active-active or active-passive
Global load balancer / DNS routing
Data replication strategy:
eventual vs strong consistency
Failover automation
? Trade-off:
Consistency vs availability (CAP theorem)

**10.** 10. Scenario: Database is the bottleneck in scaling
Answer:
Short-term:
Add read replicas
Optimize queries
Long-term:
Caching layer (Redis)
Sharding
Move to NoSQL (if applicable)
? SME insight:
Scaling apps is easy, scaling databases is hard

**11.** 11. What is toil and how do you reduce it at org level?
Answer:
Toil = manual, repetitive, low-value work
Reduction:
Automation (scripts, pipelines)
Self-healing systems
Platform engineering approach
Shift-left reliability
? Goal:
Engineers should spend time on engineering, not operations

**12.** 12. Scenario: Kubernetes cluster is healthy but app is down
Answer:
Check:
Readiness probe failures
Service/Ingress routing
Config issues (env vars/secrets)
Dependency failures
? Insight:
Infra healthy ≠ application healthy

**13.** 13. How do you approach capacity planning?
Answer:
Analyze historical usage
Identify peak patterns
Define headroom (e.g., 30%)
Use load testing
Combine with autoscaling
? SME touch:
Plan for failures + spikes, not just average load

**14.** 14. Scenario: Sudden traffic spike (10x)
Answer:
Immediate:
Enable autoscaling
Rate limiting
Serve cached responses
Protect system:
Queue requests
Load shedding
? Long-term:
Improve scaling strategy
Add CDN / caching layers

**15.** 15. How do you run effective postmortems?
Answer:
Blameless culture
Focus on:
What happened
Why it happened
What will prevent it
Action items with owners
Track recurring patterns
? SME mindset:
Postmortems are for learning, not blaming
What are SLI, SLO, and SLA?
Answer:
SLI (Indicator): Metric → e.g., request success rate
SLO (Objective): Target → 99.9% uptime
SLA (Agreement): Business contract → penalties if violated
? Example:
SLI: API success rate
SLO: 99.9%
SLA: 99.5% (customer-facing)
What is an error budget?
Answer:
Error Budget = Allowed failure = 100% - SLO
Example:
SLO = 99.9%
Error budget = 0.1% downtime
? If budget is exhausted:
Stop deployments
Focus on stability
What is toil?
Answer:
Manual, repetitive, non-scalable work.
Examples:
Restarting services manually
Re-running failed jobs
Goal: Automate or eliminate

**16.** 2. Monitoring & Observability
Difference between monitoring and observability?
Answer:
Monitoring → predefined metrics & alerts
Observability → ability to understand unknown failures using:
Metrics
Logs
Traces
What is MTTR, MTBF?
Answer:
MTTR → Mean Time To Repair
MTBF → Mean Time Between Failures
? SRE goal: Reduce MTTR, increase MTBF
How do you reduce MTTR?
Answer:
Better alerting (actionable alerts)
Runbooks
Auto-remediation
Proper dashboards
Incident drills
How do you design a highly available system?
Answer:
Multi-AZ deployment
Load balancer
Auto Scaling
Health checks
Stateless services
Database replication
How do you handle traffic spikes?
Answer:
Auto scaling (horizontal preferred)
Rate limiting
Caching (Redis/CDN)
Queue-based buffering (SQS/Kafka)
What is HPA?
Answer:
Horizontal Pod Autoscaler:
Scales pods based on CPU/memory/custom metrics
Difference between liveness & readiness probe?
Answer:
Liveness → restart container
Readiness → control traffic

**17.** 7. Scenario-Based Questions (SME Level)
Scenario 1: High CPU on production servers
What will you do?
Answer Approach:
Check metrics (CPU, memory, load)
Identify process causing spike
Check recent deployments
Scale out temporarily
Fix root cause
? SME touch:
Add autoscaling + alerting
Performance tuning
Scenario 2: API latency suddenly increased
Answer:
Check latency dashboards
Correlate with:
DB latency
Network
downstream services
Check recent deploy
Rollback if needed
Scenario 3: Alerts are too noisy
Answer:
Remove non-actionable alerts
Tune thresholds
Use SLO-based alerting
Group alerts
Use deduplication
Scenario 4: Deployment caused outage
Answer:
Immediate rollback
Activate incident response
Root cause analysis
Add safeguards (canary, tests)
Scenario 5: Database is bottleneck
Answer:
Optimize queries
Add indexes
Read replicas
Caching layer
Sharding (if needed)

**18.** 8. Advanced SME-Level Questions
How do you define SLO for a new service?
Answer:
Understand user journey
Identify critical endpoints
Define SLIs (latency, success rate)
Set realistic targets (based on baseline)
Align with business impact
What is cascading failure?
Answer:
Failure in one service → spreads to others.
? Prevention:
Circuit breakers
Timeouts
Bulkheads
Retries with backoff
What is backpressure?
Answer:
System slows intake when overloaded.
Example:
Queue length increases
Reject requests gracefully
How do you reduce cost while maintaining reliability?
Answer:
Right-size resources
Use spot instances
Autoscaling
Optimize storage/log retention
Use serverless where possible
How do you handle conflicts with developers?
Answer:
Focus on data (metrics/logs)
Align on SLO impact
Collaborate on solution
Avoid blame
How do you prioritize reliability vs feature delivery?
Answer:
Use error budget
If budget exhausted → stop releases
Otherwise → allow innovation
How do you design a fault-tolerant architecture on Amazon Web Services?
Answer:
Multi-AZ deployment (minimum baseline)
Use managed services:
ALB for load balancing
RDS Multi-AZ / DynamoDB
Stateless application layer
Auto Scaling Groups
Health checks + self-healing
? SME insight:
Always design assuming instance failure is normal

**19.** 2. Scenario: An EC2 Auto Scaling group is not scaling despite high CPU
Answer:
Check:
CloudWatch alarm thresholds
Scaling policy configuration
Cooldown period
Metrics delay
? Common root cause:
Alarm not triggering or misconfigured metric
? SME tip:
Always validate metric → alarm → action chain

**20.** 3. How do you secure a CI/CD pipeline in AWS CodePipeline?
Answer:
IAM least privilege roles
Artifact encryption (S3 + KMS)
Secrets via Secrets Manager / Parameter Store
Manual approval for prod
Audit via CloudTrail
? Advanced:
Use OIDC instead of long-lived credentials

**21.** 4. Scenario: Deployment pipeline is slow (takes 40+ mins)
Answer:
Identify bottleneck:
Build time?
Test stage?
Artifact upload?
Optimize:
Parallel stages
Cache dependencies
Use smaller Docker images
Incremental builds
? SME mindset:
Optimize for feedback loop speed

**22.** 5. How do you handle secrets in DevOps pipelines?
Answer:
Use AWS Secrets Manager or Parameter Store
Never store secrets in code
Rotate secrets automatically
Inject at runtime
? Anti-pattern:
Hardcoding secrets in CI/CD configs

**23.** 6. Scenario: Amazon RDS is experiencing high read latency
Answer:
Check slow query logs
Add read replicas
Enable caching (Redis)
Optimize indexes
? SME insight:
Database latency often = query design issue, not infra

**24.** 7. How do you design blue/green deployment on AWS?
Answer:
Two environments (Blue = current, Green = new)
Use:
ALB routing
Route 53 weighted routing
Switch traffic gradually
? Advantage:
Instant rollback

**25.** 8. Scenario: Amazon S3 access is failing intermittently
Answer:
Check:
IAM policies
Bucket policy
VPC endpoint config
Validate:
DNS resolution
Request rate limits
? SME thinking:
Intermittent issues often = network or throttling

**26.** 9. How do you optimize AWS cost without impacting reliability?
Answer:
Use:
Spot instances (non-critical workloads)
Savings Plans / Reserved Instances
Right-size resources
Auto scale down during low traffic
Lifecycle policies for S3
? Key:
Cost optimization should not break SLOs

**27.** 10. Scenario: AWS Lambda is timing out frequently
Answer:
Check:
Execution time vs timeout
Memory allocation
Increase memory (also increases CPU)
Optimize code / reduce external calls
Use async processing if needed
? SME insight:
Lambda issues are often performance-bound, not infra-bound

**28.** 11. How do you design observability in AWS?
Answer:
Metrics: CloudWatch
Logs: CloudWatch Logs
Tracing: X-Ray
Dashboards: Grafana
? Advanced:
Correlate logs + traces + metrics

**29.** 12. Scenario: Containerized app on Amazon EKS is unstable under load
Answer:
Check:
Pod resource limits
HPA config
Node capacity
Tune:
Requests/limits
Autoscaling policies
? SME insight:
Most issues = wrong resource sizing

**30.** 13. How do you ensure zero-downtime deployments?
Answer:
Rolling updates
Readiness probes
Connection draining
Canary deployments
? Important:
Always validate before shifting traffic

**31.** 14. Scenario: Logs are increasing massively, causing high cost
Answer:
Reduce log verbosity
Use sampling
Set retention policies
Filter unnecessary logs
? SME thinking:
Logs should be useful, not noisy

**32.** 15. How do you manage infrastructure at scale?
Answer:
Infrastructure as Code:
Terraform / CloudFormation / CDK
Modular design
Version control
Automated validation
? SME mindset:
Infra should be reproducible, not manual

**33.** 1. How do you design a scalable CI/CD platform for multiple teams?
Answer:
Centralized platform with:
Shared pipelines/templates
Self-service onboarding
Use:
Pipeline-as-code
Artifact repository (Nexus/Artifactory)
Isolate workloads using runners/agents
? SME insight:
Treat CI/CD as a product (platform engineering mindset)

**34.** 2. Scenario: CI pipeline failures increased after scaling teams
Answer:
Likely causes:
Shared resource contention
Flaky tests
Dependency conflicts
Fix:
Parallelize builds
Isolate environments (per-branch builds)
Introduce caching
Stabilize flaky tests
? SME thinking:
Growth exposes pipeline design flaws

**35.** 3. How do you ensure reliability of CI/CD pipelines?
Answer:
Retry mechanisms (idempotent steps)
Pipeline observability (metrics + logs)
Fail fast strategy
Versioned pipelines
Test pipelines themselves
? Key:
Pipelines are production systems, treat them the same way

**36.** 4. Scenario: Deployment succeeds but application breaks in production
Answer:
Root causes:
Environment mismatch
Config issues
Missing backward compatibility
Fix:
Add:
Pre-prod validation
Contract testing
Feature flags
? SME insight:
“Successful deployment ≠ successful release”

**37.** 5. What is GitOps and why is it important?
Answer:
Git = single source of truth
Changes via pull requests
Automated sync to cluster
Tools:
ArgoCD / Flux
? Benefits:
Auditability
Rollback via Git
Consistency

**38.** 6. Scenario: Kubernetes deployment stuck in CrashLoopBackOff
Answer:
Check:
kubectl logs
kubectl describe pod
Possible causes:
App crash
Config error
Dependency failure
? SME approach:
Don't just restart → find root cause

**39.** 7. How do you manage configuration across environments?
Answer:
Use:
ConfigMaps & Secrets
Environment overlays (Helm/Kustomize)
Avoid:
Hardcoded configs
? SME tip:
Config should be externalized and version-controlled

**40.** 8. Scenario: Kubernetes cluster nodes are underutilized but costs are high
Answer:
Likely over-provisioning
Fix:
Tune:
Resource requests/limits
Use:
Cluster autoscaler
Bin-packing strategy
? Insight:
Cost inefficiency = poor resource planning

**41.** 9. How do you secure Kubernetes clusters?
Answer:
RBAC (least privilege)
Network policies
Pod security standards
Secrets management (no plain text)
Image scanning
? Advanced:
Use OPA/Gatekeeper for policy enforcement

**42.** 10. Scenario: Rolling deployment caused downtime
Answer:
Possible issues:
No readiness probe
Too aggressive rollout
Insufficient replicas
Fix:
Add:
Readiness probes
Pod disruption budgets
Gradual rollout
? SME thinking:
Kubernetes gives tools, but misconfiguration causes outages

**43.** 11. How do you handle stateful applications in Kubernetes?
Answer:
Use StatefulSets
Persistent Volumes
Backup strategy
? Important:
Stateless is easy, stateful needs careful design

**44.** 12. Scenario: CI/CD pipeline is a bottleneck for releases
Answer:
Improve:
Parallel execution
Test optimization
Pipeline modularization
? SME insight:
Slow pipelines reduce developer productivity

**45.** 13. How do you implement progressive delivery?
Answer:
Canary deployments
Feature flags
A/B testing
? Tools:
Argo Rollouts, Flagger
? Key:
Reduce blast radius of failures

**46.** 14. Scenario: Microservices communication failures in Kubernetes
Answer:
Check:
Service discovery (DNS)
Network policies
Timeouts/retries
? Advanced:
Use service mesh (Istio)
? Insight:
Microservices fail due to network complexity

**47.** 15. How do you standardize DevOps practices across teams?
Answer:
Create:
Golden paths (templates)
Shared libraries
Enforce:
Best practices via pipelines
Enable:
Self-service platforms
? SME mindset:
Standardization improves speed + reliability

**48.** 1. SLI / SLO / SLA

**49.** Q1: How do you choose meaningful SLIs for a user-facing API?
A: Focus on user-perceived success → latency (p95/p99), error rate, availability. Avoid infra metrics like CPU unless directly impacting user experience.

**50.** Q2: What's a bad SLO?
A: One that is:
Too easy (always met)
Too strict (always violated)
Not tied to user impact

**51.** Q3: How do you handle different SLOs for different tiers?
A:
Tier-1 → 99.9%+
Internal tools → 99%
Align with business criticality

**52.** Q4: Difference between SLA vs SLO?
A:
SLO = internal target
SLA = external contractual commitment (with penalties)
? 2. Error Budget Governance

**53.** Q1: How do you calculate error budget?
A: If SLO = 99.9%, error budget = 0.1% failure allowed

**54.** Q2: What happens when error budget is exhausted?
A:
Freeze releases
Focus on reliability fixes
Leadership escalation

**55.** Q3: How do you make teams respect error budgets?
A:
Tie to release velocity
Integrate into CI/CD
Visible dashboards

**56.** Q4: Can error budgets be flexible?
A: Yes — during critical business events (e.g., launch), but must be tracked and justified
? 3. Multi-Region Architecture

**57.** Q1: Active-Active vs Active-Passive?
A:
Active-Active → low latency, complex consistency
Active-Passive → simpler, slower failover

**58.** Q2: How do you handle data consistency?
A:
Use eventual consistency
Conflict resolution strategies

**59.** Q3: What are failure domains?
A:
AZ, region, service dependencies

**60.** Q4: How do you test failover?
A:
GameDays
Simulate region outages
? 4. Scenario – High Latency Spike

**61.** Q1: First dashboard you check?
A: Golden signals dashboard

**62.** Q2: How do you isolate root cause?
A:
Correlate metrics + traces
Narrow down service dependency

**63.** Q3: What if DB is bottleneck?
A:
Add read replicas
Optimize queries

**64.** Q4: Prevent future incidents?
A:
Add SLO alerts
Capacity planning
? 5. Observability Strategy

**65.** Q1: Difference between monitoring vs observability?
A:
Monitoring = known issues
Observability = unknown issues debugging

**66.** Q2: Why distributed tracing?
A:
Track request across services
Identify latency bottlenecks

**67.** Q3: What are RED metrics?
A:
Rate, Errors, Duration

**68.** Q4: How do you implement correlation?
A:
Use trace IDs across logs, metrics, traces
? 6. Scenario – Alert Noise

**69.** Q1: What is alert fatigue?
A: Too many alerts → ignored signals

**70.** Q2: How do you fix it?
A:
Remove non-actionable alerts
Use SLO-based alerts

**71.** Q3: What is good alert?
A:
Actionable
Indicates user impact

**72.** Q4: Tools used?
A:
PagerDuty
Opsgenie
? 7. CI/CD Reliability Gates

**73.** Q1: What are reliability gates?
A:
Checks before deployment (tests, SLOs)

**74.** Q2: Canary deployment advantage?
A:
Limits blast radius

**75.** Q3: What is progressive delivery?
A:
Gradual rollout with monitoring

**76.** Q4: How to automate rollback?
A:
Based on metrics breach
? 8. Scenario – Deployment Causing Incidents

**77.** Q1: First action?
A: Rollback

**78.** Q2: How to detect faster?
A:
Post-deploy monitoring
Synthetic tests

**79.** Q3: Prevent recurrence?
A:
Improve testing
Add canary

**80.** Q4: Who owns it?
A:
Shared ownership (Dev + SRE)
? 9. Kubernetes Reliability

**81.** Q1: What ensures pod stability?
A:
Liveness & readiness probes

**82.** Q2: What is HPA?
A:
Auto-scales pods based on metrics

**83.** Q3: How to avoid noisy neighbor issue?
A:
Resource limits/requests

**84.** Q4: What is PDB?
A:
Ensures minimum pods during disruption
? 10. Scenario – Pod CrashLoopBackOff

**85.** Q1: Common causes?
A:
App crash
Wrong config
OOM

**86.** Q2: Debug steps?
A:
kubectl logs, events

**87.** Q3: What is OOMKilled?
A:
Memory limit exceeded

**88.** Q4: Prevent it?
A:
Right resource sizing
? 11. Incident Management

**89.** Q1: What defines severity?
A:
User/business impact

**90.** Q2: What is MTTR?
A:
Mean time to recovery

**91.** Q3: What is blameless culture?
A:
Focus on system, not people

**92.** Q4: Tools?
A:
PagerDuty
? 12. RCA / Postmortem

**93.** Q1: What is a good RCA?
A:
Timeline + root cause + action items

**94.** Q2: What is 5 Whys?
A:
Iterative root cause analysis

**95.** Q3: Common mistake?
A:
Blaming individuals

**96.** Q4: How to track actions?
A:
Jira / backlog
? 13. Chaos Engineering

**97.** Q1: Why needed?
A:
Validate resilience

**98.** Q2: Example experiment?
A:
Kill pods / inject latency

**99.** Q3: Risks?
A:
Production impact

**100.** Q4: Tool?
A:
Chaos Monkey
? 14. Capacity Planning

**101.** Q1: How forecast load?
A:
Historical + trend analysis

**102.** Q2: What is headroom?
A:
Extra capacity buffer

**103.** Q3: Vertical vs horizontal scaling?
A:
Vertical = bigger instance
Horizontal = more instances

**104.** Q4: When auto-scaling fails?
A:
Sudden spikes / misconfigured thresholds
? 15. Scenario – Traffic Surge

**105.** Q1: First preparation?
A: Load testing

**106.** Q2: Key components?
A:
CDN
Auto-scaling

**107.** Q3: DB scaling?
A:
Read replicas

**108.** Q4: Risk?
A:
Thundering herd problem
? 16. Cost Optimization

**109.** Q1: Biggest cost leak?
A: Idle resources

**110.** Q2: Spot vs Reserved?
A:
Spot = cheap, interruptible
Reserved = predictable

**111.** Q3: How to track cost?
A:
AWS Cost Explorer

**112.** Q4: FinOps role?
A:
Align cost with business value
? 17. Scenario – High Cloud Bill

**113.** Q1: First step?
A: Cost breakdown analysis

**114.** Q2: Common causes?
A:
Over-provisioning
Data transfer

**115.** Q3: Fix?
A:
Rightsizing
Auto-scaling

**116.** Q4: Prevent?
A:
Budget alerts
? 18. Infrastructure as Code

**117.** Q1: Why Terraform?
A:
Declarative, reusable

**118.** Q2: What is drift?
A:
Manual changes vs code mismatch

**119.** Q3: How detect drift?
A:
terraform plan

**120.** Q4: Best practice?
A:
Modular design
? 19. Scenario – Region Failure

**121.** Q1: RTO vs RPO?
A:
RTO = recovery time
RPO = data loss tolerance

**122.** Q2: Failover method?
A:
DNS routing

**123.** Q3: Data replication types?
A:
Sync / Async

**124.** Q4: Testing DR?
A:
Regular drills
? 20. Leadership & Influence

**125.** Q1: How influence teams?
A:
Show metrics impact

**126.** Q2: Handle resistance?
A:
Educate + align with business goals

**127.** Q3: What is platform engineering?
A:
Internal developer platform

**128.** Q4: Measure success?
A:
MTTR, SLO compliance, deployment success